2026-5-21 Linux常见指令解析：下

- 11. more 指令
  - 11.1 功能与语法
  - 11.2 控制显示行数
  - 11.3 手记补充
- 12. less 指令
  - 12.1 功能与语法
  - 12.2 常用操作
  - 12.3 常用选项
  - 12.4 与 more 的区别
- 13. head 指令
  - 13.1 功能与语法
  - 13.2 指定行数
  - 13.3 易错点
- 14. tail 指令
  - 14.1 功能与语法
  - 14.2 常用选项
  - 14.3 截取文件中间的一段
  - 14.4 手记补充
- 15. date 指令
  - 15.1 功能与语法
  - 15.2 Unix 时间戳
  - 15.3 设置时间
  - 15.4 易错点
- 16. cal 指令
  - 16.1 功能与语法
  - 16.2 常用选项
- 17. find 指令
  - 17.1 功能与语法
  - 17.2 示例
  - 17.3 为什么 find 可能较慢
- 18. which 指令
  - 18.1 功能与语法
  - 18.2 which 与 type
  - 18.3 手记补充
- 19. whereis 指令

    - 19.1 功能与语法
    - 19.2 与 which、find 的区别
- 20. file 指令
    - 20.1 功能与语法
    - 20.2 常用选项
    - 20.3 手记补充
- 21. alias 指令
    - 21.1 功能与语法
    - 21.2 查看与取消别名
    - 21.3 手记补充
- 22. grep 指令
    - 22.1 功能与语法
    - 22.2 常用选项
    - 22.3 管道中的 grep
    - 22.4 手记补充
- 23. top 指令
    - 23.1 功能与语法
    - 23.2 观察重点
- 24. zip 和 unzip 指令
    - 24.1 打包和压缩不是同一件事
    - 24.2 zip
    - 24.3 unzip
- 25. rz 和 sz 指令
    - 25.1 功能
    - 25.2 安装
- 26. tar 指令
    - 26.1 功能
    - 26.2 常用选项
    - 26.3 创建归档
    - 26.4 查看归档内容
    - 26.5 解压归档
    - 26.6 常见高级用法
    - 26.7 易错点
- 27. bc 和 uname 指令
    - 27.1 bc
    - 27.2 uname
- 28. 常用快捷键与安全关机

- 28.1 终端快捷键
- 28.2 关机与重启

- 29. 命令收束

11. more 指令

11.1 功能与语法

`more` 用来分页查看文本文件。与 `cat` 一次性把内容输出到终端不同，`more` 会在每屏内容后暂停，适合查看稍长的文本。

```
more [选项] [文件]
```

```
more temp.txt
```

进入 `more` 后，常用操作如下：

按键或输入	作用
`Space`	向下翻一屏
`Enter`	向下移动一行
`q`	退出

11.2 控制显示行数

有的 `more` 实现可以直接写成 `more -10 file`，表示每屏显示 10 行：

```
more -10 temp.txt
```

不同发行版的 `more` 对参数支持不完全一致。如果 `-10` 不生效，可以先查看本机帮助：

```
man more
```

`more` 的主要限制是一般只能向下浏览，不能像现代编辑器那样自由向上翻页。因此实际查看长文件时，通常更推荐 `less`。

11.3 手记补充

课堂手记强调：`more` 的核心是“分页”，不是修改文件。把它和 `cat` 比较时，最直观的差异是 `cat` 适合短文本，`more` 适合内容较多、需要逐屏阅读的场景。

12. less 指令

12.1 功能与语法

`less` 也是分页查看工具，功能通常比 `more` 完整。它可以在文件中前后翻动、搜索内容，并且不必先把整个文件加载到内存中，因此适合查看大文件。

```
less [选项] 文件
```

```
less temp.txt
```

12.2 常用操作

操作	作用
`Space` 或 `PageDown`	向下翻页
`b` 或 `PageUp`	向上翻页
`/字符串`	向下搜索
`?字符串`	向上搜索
`n`	重复上一次搜索
`N`	反向重复上一次搜索
`q`	退出

12.3 常用选项

选项	作用
`-N`	显示行号

选项	作用
`-i`	搜索时忽略大小写

```
less -N temp.txt
```

搜索时可以先输入 `/`，再输入关键字并回车：

```
/error
```

`n` 会继续查找同一个关键字，`N` 会反向查找。

12.4 与 more 的区别

`more` 更简单，通常只能向后浏览；`less` 可以前后翻看、搜索，并且是查看日志和大文件的常用工具。若只需要简单翻页，`more` 足够；需要检索和反复定位时，用 `less`。

13. head 指令

13.1 功能与语法

`head` 用来显示文件开头的内容。默认显示前 10 行。

```
head [选项] [文件]
```

```
head temp.txt
```

13.2 指定行数

用 `-n` 指定行数，也可以简写成 `-数字`：

```
head -n 5 temp.txt
head -5 temp.txt
```

`head` 通常不会修改文件，只把选中的内容输出到终端。与管道结合时，它经常用于截取一段数据：

```
head -20 temp.txt | tail -5
```

这段命令先取前 20 行，再取这 20 行中的最后 5 行，因此得到第 16-20 行。

13.3 易错点

`head` 和 `tail` 的默认单位是“行”，不是“字节”。如果需要按字节读取，应查看对应版本的 `man head`，使用相应的字节选项。

14. tail 指令

14.1 功能与语法

`tail` 用来查看文件末尾内容，默认显示最后 10 行。

```
tail [选项] [文件]
```

```
tail temp.txt
```

14.2 常用选项

选项	作用
`-n 行数`	指定显示末尾多少行
`-f`	持续跟踪文件变化，常用于查看日志

```
tail -n 3 temp.txt
tail -f app.log
```

查看动态日志时，`tail -f` 会持续刷新输出。按 `Ctrl+C` 可以停止跟踪并回到提示符。

14.3 截取文件中间的一段

最常见的组合是“先用 head 截到末尾位置，再用 tail 取需要的长度”：

```
head -200 temp.txt | tail -20
```

这条命令得到第 181-200 行。若希望包含第 180 行，应把最后的行数改成 21：

```
head -200 temp.txt | tail -21
```

这里的 `|` 是管道，表示把左边命令的标准输出交给右边命令作为标准输入。它不是重定向到文件。

14.4 手记补充

`tail -f` 的价值不只在“看最后几行”，而在于持续观察正在写入的日志。实际查看服务日志时，还要注意日志轮转、权限和文件被替换等情况。

15. date 指令

15.1 功能与语法

`date` 用来显示或设置系统时间，也可以按指定格式输出时间。

```
date [选项] [+格式]
```

```
date
date "+%Y-%m-%d %H:%M:%S"
```

常用格式如下：

格式	含义
`%Y`	四位年份
`%m`	月份 `01-12`
`%d`	日期 `01-31`
`%H`	小时 `00-23`
`%M`	分钟 `00-59`

格式	含义
`%S`	秒 `00-59`
`%F`	等价于 `%Y-%m-%d`
`%T`	等价于 `%H:%M:%S`
`%s`	Unix 时间戳，单位为秒

15.2 Unix 时间戳

Unix 时间戳是从 1970 年 1 月 1 日 00:00:00 UTC 起经过的秒数。它本质上是一个不断增长的计数器，适合写入日志、监控数据或传给程序。

```
date +%s
```

时间戳转成可读时间：

```
date -d @0
date -d @1000000000 "+%Y-%m-%d %H:%M:%S"
```

由于本机时区不同，`@0` 显示出来的本地时间可能不是 `00:00:00`。例如在东八区，它通常显示为 1970-01-01 08:00:00。

可读时间转为时间戳：

```
date -d "2026-10-27 12:00:00" +%s
```

时间戳适合程序处理，可读时间适合人阅读。手记中提到“日志不想直接输出年月日、时间给计算机看、监控需要时间戳”，说的正是这两种表示方式的用途分工。

15.3 设置时间

设置时间通常需要 `root` 权限：

```
sudo date -s "2026-10-27 12:00:00"
```

实际生产环境通常应由时间同步服务维护系统时间，不建议随意手动修改，尤其不要用 `date -s` 代替时间同步。

15.4 易错点

`date` 的格式串要放在引号中，否则 Shell 可能解释其中的空格或特殊字符：

```
date "+%Y/%m/%d - %H:%M:%S"
```

16. cal 指令

16.1 功能与语法

`cal` 用来显示公历日历。

```
cal [选项] [月份] [年份]
```

```
cal
cal 2026
cal 10 2026
```

只有一个数字参数时，通常把它解释为年份；提供两个数字参数时，通常先月份、后年份。实际格式以本机 `man cal` 为准。

16.2 常用选项

选项	作用
`-3`	显示前一个月、当前月、下一个月
`-y`	显示当前年份的日历
`-j`	显示一年中的第几天

```
cal -3
cal -y
cal -j
```

`cal` 只是显示工具，不会修改系统日期。部分精简系统可能默认没有安装它，可以按发行版使用 `apt` 或 `yum` 安装。

17. find 指令

17.1 功能与语法

`find` 会在指定的目录树中递归查找文件，并按照条件执行操作。

```
find [搜索路径] [表达式]
```

```
find . -name "test.c"
find /home -type f -name "*.txt"
```

常用表达式：

表达式	作用
`-name 模式`	按文件名匹配
`-type f`	只查找普通文件
`-type d`	只查找目录
`-size +10M`	查找大于 10M 的文件
`-mtime -1`	查找最近一天内修改的文件

17.2 示例

在当前目录及子目录中查找 C 源文件：

```
find . -type f -name "*.c"
```

在家目录中查找名为 `test.c` 的文件：

```
find ~ -name "test.c"
```

查找当前目录下超过 10 MB 的文件：

```
find . -type f -size +10M
```

匹配通配符时建议给模式加引号，避免 Shell 在 `find` 执行前先把通配符展开。

17.3 为什么 find 可能较慢

`ls` 通常只读取一个目录的目录项；`find` 要沿着目录树逐层遍历，可能访问大量目录和文件。在很大的文件系统上，查找成本会明显增加。

`find` 的本质是去文件树中寻找匹配项，不一定只查一个固定目录。手记中所说的“命令通常是可执行文件，软件安装往往就是把程序放到系统路径中”，讲的是命令文件的位置；查找文件本身仍然要遵循 `find` 的遍历规则。

18. which 指令

18.1 功能与语法

`which` 用来显示在当前 `PATH` 中匹配到某个命令的可执行文件路径。

```
which 命令名
```

```
which ls
which pwd
```

输出可能是：

```
/usr/bin/ls
/usr/bin/pwd
```

注意系统目录通常是 `/usr/bin`，不是 `/user/bin`。手记中把路径抄成 `/user/bin`，实际使用时应以 `which` 和 `type` 的输出为准。

18.2 which 与 type

`which` 关心的是“在路径中找到了哪个可执行文件”，但 `cd`、`alias` 等可能由 Shell 内建或扩展处理。想确认某个名字在当前 Shell 中究竟如何解释，可以使用：

```
type ls
type cd
type ll
```

`type` 会告诉你是别名、Shell 内建命令还是外部程序，通常比只看 `which` 更完整。

18.3 手记补充

自己写的程序不建议随意复制到 `/usr/bin` 等系统命令目录，以免覆盖或污染系统命令环境。运行当前目录中的程序时，明确使用：

```
./mycmd
```

19. whereis 指令

19.1 功能与语法

`whereis` 用来查找程序的相关文件，通常会报告二进制文件、源代码和手册页所在位置。

```
whereis [选项] 名称
```

```
whereis ls
whereis libc.so
```

典型输出可能包含：

```
ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz
```

19.2 与 which、find 的区别

命令	搜索范围	主要用途
`which`	`PATH` 中的命令	查看实际调用的命令路径
`whereis`	预设的系统目录	查程序、手册页和源码位置
`find`	指定目录树	按条件进行广泛查找

`whereis` 不会像 `find /` 那样遍历整个文件系统，也不会搜索普通用户任意创建的文本文件。它的速度较快，但结果范围也更窄。

20. file 指令

20.1 功能与语法

`file` 用来判断文件的实际类型。

```
file [选项] 文件或目录...
```

```
file hello.c
file my.exe
file temp.txt
```

输出可能类似：

```
hello.c: C source, ASCII text
temp.txt: ASCII text
```

20.2 常用选项

选项	作用
`-b`	不显示文件名，只显示识别结果
`-i`	显示 MIME 类型
`-z`	尝试识别压缩文件中的内容

```
file -b temp.txt
file -i temp.txt
file -z archive.tar.gz
```

20.3 手记补充

Linux 不会只凭 `.exe`、`.txt` 这类扩展名决定文件类型。`file` 会读取文件内容中的特征，因此它可以指出一个文件到底是文本、脚本、可执行程序还是压缩数据。

“文件类型”和“执行权限”是两件事：

```
file my.exe
ls -l my.exe
```

即使 `file` 识别出它是可执行文件，只要缺少执行权限，仍然不能直接运行。执行权限相关操作将在后续权限专题中展开。

21. alias 指令

21.1 功能与语法

`alias` 用来给命令设置别名，减少重复输入。

```
alias 名称='命令'
```

```
alias hello='ls -aln'
hello
```

等号两边不能随意加空格；命令部分要使用引号，避免 Shell 提前拆分。

21.2 查看与取消别名

```
alias
alias hello
unalias hello
```

`alias` 的输出通常保存在当前 Shell 会话中。关闭终端或重新登录后，临时的别名可能消失；若希望永久生效，需要把它写入 Shell 的启动配置，例如 Bash 的 `~/.bashrc`。

21.3 手记补充

别名只是 Shell 层的名字映射，不会在 `/usr/bin` 中创建新的程序文件。Shell 执行命令时可能优先解析别名，因此看到同名命令时要先检查：

```
type hello
```

不要把重要命令随意改成含义差别很大的别名，也不建议用别名覆盖系统常用命令，以免在脚本或他人环境中行为不一致。

22. grep 指令

22.1 功能与语法

`grep` 用来在文本中按行查找字符串，并把包含匹配内容的行打印出来。默认区分大小写。

```
grep [选项] 搜索字符串 文件...
```

```
grep "error" app.log
grep "error" *.log
```

22.2 常用选项

选项	作用
`-i`	忽略大小写
`-n`	显示匹配行的行号
`-v`	反向选择，显示不含关键字的行
`-c`	只输出匹配行数
`-r`	递归搜索目录中的文件

```
grep -i "error" app.log
grep -n "error" app.log
grep -v "debug" app.log
grep -rn "TODO" ./src
```

22.3 管道中的 grep

`grep` 不仅可以读取文件，也可以读取另一个命令的输出：

```
ps aux | grep ssh
cat app.log | grep -in "failed"
```

第二种写法可以直接写成：

```
grep -in "failed" app.log
```

管道适合在已有命令输出的基础上继续过滤；如果本来就有一个文件，直接指定文件名通常更清楚。

22.4 手记补充

`grep` 是行文本处理工具，适合过滤错误日志、查找配置关键字，也可以把 `ps` 等命令的输出再缩小范围。它默认大小写敏感，需要忽略大小写时显式加上 `-i`。

23. top 指令

23.1 功能与语法

`top` 用来动态查看系统和进程运行状态，可以理解为终端中的任务管理器。

```
top [选项]
```

```
top
top -d 1 -n 5
```

`-d` 指定刷新间隔，单位为秒；`-n` 指定刷新次数。上面的例子表示每秒刷新一次，共采集 5 次。

23.2 观察重点

`top` 页面上通常包括：

- 系统运行时间和负载。
- CPU、内存和交换空间使用情况。
- 进程 PID、用户、CPU 占用、内存占用和状态。
- 当前占用资源较高的进程。

常用操作：

操作	作用
`q`	退出
`P`	按 CPU 使用率排序
`M`	按内存使用率排序
`k`	向进程发送信号，操作前要确认 PID

查看进程状态时，不要只看到高 CPU 就立即结束进程；先确认真实 PID、用户、进程用途和启动命令。

24. zip 和 unzip 指令

24.1 打包和压缩不是同一件事

- 打包：把多个文件或目录合成一个归档文件。
- 压缩：通过算法减少数据占用的存储空间。

打包主要解决“文件太多、传输和保存不方便”的问题；压缩主要解决“文件体积大、传输慢”的问题。实际使用中两者经常一起完成。

课堂手记中提到，压缩本质上是利用本地 CPU 换取更小的网络传输量：发送前压缩，接收后解压，通常能减少传输时间和带宽占用。

24.2 zip

```
zip [选项] 压缩文件.zip 文件或目录
```

压缩文件：

```
zip backup.zip file.txt
```

递归压缩目录及其子目录：

```
zip -r backup.zip dir/
```

如果目录中包含子目录，必须加 `-r`，否则可能只打包部分内容或跳过子目录。

24.3 unzip

```
unzip 压缩文件.zip [-d 目标目录]
```

```
unzip backup.zip
unzip backup.zip -d /tmp/backup
unzip -l backup.zip
```

`-l` 只列出压缩包内容，不实际解压。解压前先查看内容，可以避免覆盖意外文件。

25. rz 和 sz 指令

25.1 功能

`rz` 和 `sz` 通常由 `lrzsz` 工具提供，用于在支持 ZMODEM 的终端与远端 Linux 主机之间传输文件：

- `rz`：从本地终端上传文件到远端 Linux。
- `sz`：从远端 Linux 下载文件到本地终端。

它们适合通过 XShell 等终端工具临时传输文件，不是普通的 Linux 网络文件共享协议。

25.2 安装

按发行版选择包管理器：

```
sudo apt install -y lrzsz
```

```
sudo yum install -y lrzsz
```

安装后可以运行：

```
rz
sz file.txt
```

不同终端的 ZMODEM 支持不同，部分终端需要手动启用。若终端不支持弹窗选择文件，仍然可以通过其他文件传输方式完成上传下载。

26. tar 指令

26.1 功能

`tar` 用于打包和解包，也可以配合 `gzip` 或 `bzip2` 完成压缩。它最常见的任务有：

- 创建归档文件。
- 查看归档文件内容。
- 从归档文件提取全部或部分文件。
- 在解压时保留权限。

26.2 常用选项

选项	作用
`-c`	create，创建归档
`-x`	extract，提取归档
`-t`	list，查看归档内容
`-z`	使用 gzip 压缩或解压
`-j`	使用 bzip2 压缩或解压
`-v`	显示处理过程
`-f`	指定归档文件名
`-C`	提取到指定目录
`-p`	提取时保留原权限等信息

26.3 创建归档

只打包，不压缩：

```
tar -cvf etc.tar /etc
```

使用 gzip 压缩：

```
tar -zcvf etc.tar.gz /etc
```

使用 bzip2 压缩：

```
tar -jcvf etc.tar.bz2 /etc
```

`-f` 后面要紧接归档文件名。常见写法会把选项合在一起，例如 `-zcvf`，但归档名必须紧跟在这组选项后面。

26.4 查看归档内容

```
tar -tf etc.tar
tar -ztvf etc.tar.gz
```

先查看再解压，能确认归档中是否包含绝对路径、深层目录或意外文件。

26.5 解压归档

解压到当前目录：

```
tar -zxvf etc.tar.gz
```

解压到指定目录：

```
tar -zxvf etc.tar.gz -C /usr/local/src
```

只提取归档中的一个文件：

```
tar -zxvf etc.tar.gz etc/passwd
```

需要保留原权限时可以使用 `-p`：

```
tar -zxvpf etc.tar.gz
```

26.6 常见高级用法

打包时排除某个目录：

```
tar --exclude='home/dmtsai' -zcvf home.tar.gz /home
```

排除模式匹配的是归档中的成员名。归档时开头的 `/` 通常会被去掉，所以这里应写成 `home/dmtsai`，而不是直接照搬绝对路径 `/home/dmtsai`。

只备份指定日期之后修改的文件：

```
tar -N "2026-10-01" -zcvf home.tar.gz /home
```

通过管道把归档内容直接交给另一个 tar 解包：

```
tar -cvf - /etc | tar -xvf -
```

这里的第一个 `-` 表示标准输出，第二个 `-` 表示标准输入，中间的 `|` 是管道。该写法不产生中间归档文件，常用于将目录内容直接复制到另一个位置。

26.7 易错点

> ⚠ **不要混用创建和解压参数**
>
> 创建归档使用 `-c`，提取归档使用 `-x`，查看内容使用 `-t`。`-zcvf` 是“创建 gzip 归档”，不是“解压”。

绝对路径在归档中可能触发有关前导 `/` 的警告。为防止把文件解到错误的位置，解压前应先 `tar -tf` 查看内容，并使用 `-C` 明确指定目标目录。

27. bc 和 uname 指令

27.1 bc

`bc` 是一个命令行计算器，支持整数、浮点数和较高精度计算。

```
bc
```

进入交互界面后输入：

```
scale=2
10/3
quit
```

也可以直接在命令中计算：

```
echo "scale=2; 10/3" | bc
```

`scale` 控制小数位数。使用 `bc -l` 可以加载数学库，适合正弦、对数等更复杂的计算：

```
echo "s(1)" | bc -l
```

27.2 uname

`uname` 用来查看系统和内核信息：

```
uname -r
uname -a
```

- `uname -r`：查看内核版本。
- `uname -a`：显示内核名、主机名、版本、硬件平台等信息。

查看发行版信息时，可以尝试：

```
lsb_release -a
cat /etc/os-release
```

不同发行版提供的命令不完全一致，`/etc/os-release` 通常是更通用的查看入口。

28. 常用快捷键与安全关机

28.1 终端快捷键

快捷键	作用
`Tab`	命令或文件路径补全
`Ctrl+C`	中止当前前台程序
`Ctrl+D`	表示输入结束，也可用于退出某些交互程序
`Ctrl+L`	清屏，通常等同于 `clear`

`Ctrl+C` 是发送中断信号，不是复制。XShell 中复制粘贴常使用 `Ctrl+Insert` 和 `Shift+Insert`，具体以终端设置为准。

28.2 关机与重启

需要系统管理权限：

```
shutdown -h now
shutdown -r now
shutdown -h +10
```

- `-h`：关机，停止系统服务后关闭机器。
- `-r`：重启。
- `+10`：10 分钟后执行。

不要在有其他用户登录或关键服务运行时随手执行关机命令。正式系统中应使用 `systemctl`、云平台控制台或管理员规定的运维流程。

29. 命令收束

任务	命令示例
分页查看文件	`less file`
查看文件开头	`head -n 20 file`
查看文件末尾	`tail -n 20 file`
持续查看日志	`tail -f app.log`
显示当前时间	`date "+%F %T"`
时间戳转换	`date -d @1000000000`

任务	命令示例
查看日历	`cal -3`
在目录树中查找	`find . -name "*.c"`
查找命令路径	`which ls`
查找程序相关文件	`whereis ls`
判断文件类型	`file data.bin`
设置临时别名	`alias ll='ls -al'`
文本过滤	`grep -in "error" app.log`
查看进程状态	`top`
压缩目录	`zip -r backup.zip dir/`
解压 zip	`unzip backup.zip -d target/`
创建 tar.gz	`tar -zcvf backup.tar.gz dir/`
查看 tar 内容	`tar -ztvf backup.tar.gz`
解压 tar.gz	`tar -zxvf backup.tar.gz -C target/`
交互式计算	`bc`
查看内核版本	`uname -r`

这一篇把“查看、查找、判断、过滤、观察进程、压缩传输”这些高频动作串了起来。复习时不要只背选项，先判断要完成的任务，再选择合适的命令；遇到不确定的参数，用 `man 命令` 查看本机实际支持的行为。